

Guion de defensa y explicación de implementación

TP3 - Sistema 1

Grupo [GXXCSS] – completar con nombres reales

Versión de apoyo para la defensa oral

Índice

1. Cómo usar este documento	3
2. Guion técnico: cómo lo implementamos	3
2.1. Modelo físico que pasamos a código	3
2.2. Por qué usamos simulación dirigida por eventos	4
2.3. Inicialización de partículas	4
2.4. Predicción de colisiones	4
2.5. Cola priorizada e invalidación diferida	4
2.6. Resolución de eventos	5
2.7. Observables que guarda el motor	5
2.8. Por qué el archivo de salida incluye color	5
2.9. Cómo se arma el estudio 1.1–1.4	6
2.10. Cómo calculamos cada observable	6
2.11. Limitaciones que conviene reconocer antes de que las marquen	7
2.12. Frases a evitar	7
3. Guion de la presentación	7
3.1. Ruta sugerida para 13 minutos	7
3.2. Slide por slide	8
3.3. Plan B si los cortan de tiempo	11
4. Preguntas y respuestas posibles	12
4.1. Implementación del motor	12
4.2. Análisis y resultados	12
4.3. Uso responsable de IA y validación	13
5. Uso responsable de IA	14

6. Checklist para una demostración en vivo	14
6.1. Secuencia recomendada	14
6.2. Comandos	15
6.3. Qué deberían poder comentar si corre bien	15
6.4. Qué no conviene hacer en vivo	15
7. Checklist final antes de exponer	15

1. Cómo usar este documento

Este PDF está pensado como material de estudio previo a la defensa, no como texto para leer en voz alta.

- **Sección 2:** resume qué hace realmente el código y cómo explicarlo sin caer en detalles irrelevantes.
- **Sección 3:** propone un recorrido slide por slide alineado con `presentation/system1-beamer/deck.tex`.
- **Sección 4:** junta preguntas probables del profesor con respuestas cortas y defendibles.
- **Sección 5:** ayuda a responder si preguntan por uso responsable de IA.
- **Sección 6:** deja una checklist corta para una demo en vivo que sí funciona en este checkout.

Idea fuerza de la defensa

Si tienen que condensar todo en cuatro mensajes, que sean estos:

1. Implementamos un **motor dirigido por eventos** porque el problema está definido por colisiones y cambios de estado puntuales, no por un Δt fijo.
2. Separarnos **motor**, **salida textual**, **animación** y **análisis**, para que cada parte tenga un contrato claro.
3. Los observables pedidos por el TP no salen “de una figura”: salen de **series temporales y perfiles radiales** que el motor preserva y el estudio procesa.
4. Las conclusiones que mostramos son **acotadas al dataset actual**: 2 realizaciones por valor de N en `artifacts/system1/studies/inciso-1.1`. No conviene sobregeneralizar.

2. Guion técnico: cómo lo implementamos

2.1. Modelo físico que pasamos a código

- El sistema es un anillo circular: recinto de diámetro $L = 80$ m, obstáculo fijo central de radio $r_0 = 1$ m y partículas de radio $r = 1$ m.
- Cada partícula tiene posición (x, y) , velocidad (v_x, v_y) , masa $m = 1$ y un estado discreto: **fresh** o **used**.
- Todas empiezan frescas. Una fresca pasa a usada al tocar el obstáculo. Una usada vuelve a fresca al tocar la pared exterior.
- Entre colisiones el movimiento es balístico: velocidad constante y trayectorias rectas.

Frase defendible: “Tomamos el modelo matemático del enunciado y lo representamos con una partícula por registro, una geometría fija compartida y tres tipos de eventos: choque entre partículas, choque con la pared y choque con el obstáculo.”

2.2. Por qué usamos simulación dirigida por eventos

- Un esquema con Δt fijo obliga a elegir un paso artificial y puede saltarse colisiones o volver cara la simulación si el paso es muy chico.
- En este problema los cambios relevantes ocurren exactamente en los tiempos de colisión.
- Por eso el motor predice cuál es el siguiente evento, avanza el sistema hasta ese tiempo y recién ahí modifica el estado.

Frase defendible: “No discretizamos el tiempo; discretizamos los eventos. Eso nos deja trabajar en tiempos físicos de colisión y evita introducir un parámetro numérico que el enunciado no pide.”

2.3. Inicialización de partículas

- La inicialización principal intenta ubicar partículas al azar por rechazo dentro de la región accesible, evitando solapamientos.
- Si esa estrategia falla por densidad o por cantidad de intentos, hay una estrategia de respaldo que arma posiciones candidatas sobre anillos internos y luego las mezcla aleatoriamente.
- Las direcciones iniciales de velocidad se toman uniformes en $[0, 2\pi)$ y el módulo vale $v_0 = 1$ m/s.

Qué muestra que entendemos el código: la inicialización no asume que el rechazo aleatorio siempre alcanza; tiene un fallback explícito para no romperse cuando el espacio libre se vuelve más difícil.

2.4. Predicción de colisiones

- **Partícula-partícula:** resolvemos la ecuación cuadrática de contacto entre centros y elegimos la primera raíz positiva físicamente válida.
- **Partícula-pared exterior:** resolvemos cuándo la trayectoria balística intersecta el radio de viaje exterior.
- **Partícula-obstáculo:** hacemos lo mismo para el radio de viaje interior.
- En los choques con fronteras circulares no alcanza con encontrar una raíz positiva: también hay que verificar que corresponda al sentido físico correcto, hacia afuera para pared y hacia adentro para obstáculo.

Frase defendible: “La predicción no es aproximada por frames; sale de resolver las condiciones geométricas de contacto.”

2.5. Cola priorizada e invalidación diferida

- Los eventos futuros se guardan en un **heap**, ordenados por tiempo.
- Cada evento guarda el tiempo predicho y los contadores de colisión de las partículas involucradas al momento de programarlo.
- Si una de esas partículas choca antes por otro motivo, el evento viejo queda obsoleto.

- No intentamos borrar esos eventos obsoletos dentro del heap. Los dejamos y los descartamos cuando salen a la cima. Eso es invalidación diferida.

Por qué importa: evita tener que reconstruir toda la cola después de cada choque y mantiene el algoritmo simple.

2.6. Resolución de eventos

- **Choque partícula-partícula:** resolvemos un choque elástico que conserva cantidad de movimiento y energía cinética del modelo.
- **Choque con pared u obstáculo:** reflejamos la velocidad respecto de la normal local.
- Encima de la colisión geométrica agregamos la lógica de estado:
 - obstáculo: si la partícula estaba fresca, se registra un contacto fresca $\rightarrow$ usada y luego queda usada;
 - pared exterior: si estaba usada, vuelve a fresca.
- Después de cada evento solo se recalculan los futuros choques de las partículas afectadas.

Frase defendible: “La dinámica mecánica y la dinámica de estados están superpuestas: primero existe el choque físico, y sobre ese choque el problema define un cambio lógico.”

2.7. Observables que guarda el motor

- $C_{fc}(t)$: cada vez que una partícula fresca toca el obstáculo, se incrementa el contador acumulado.
- $F_u(t)$: en cada snapshot contamos cuántas partículas usadas hay y dividimos por N .
- **Perfiles radiales:** en cada snapshot se recorren solo partículas frescas con $\mathbf{R} \cdot \mathbf{v} < 0$, se las binea por radio y se acumulan densidad y velocidad normal.

Esto es importante para defenderse de preguntas del tipo “*Èse gráfico lo armaron afuera o sale del simulador?*”. La respuesta correcta es: **el motor preserva la información cruda y el pipeline de estudio la agrega después.**

2.8. Por qué el archivo de salida incluye color

- El enunciado pide que la salida textual del motor imprima color.
- Por eso el `.txt` exporta no solo posición, velocidad y estado, sino también el RGB asociado.
- Eso desacopla la animación del simulador: cualquier renderer externo puede colorear sin reimplementar la lógica de estados.

Frase defendible: “Color no es una decoración del GIF; es parte del contrato de salida pedido por el TP.”

2.9. Cómo se arma el estudio 1.1–1.4

- `tp3 system1 run` corre una realización individual y escribe snapshots.
- `tp3 system1 animate` toma ese `.txt` y genera un GIF independiente.
- `tp3 system1 study` corre varias realizaciones por valor de N , guarda series crudas, agrega CSVs y genera figuras.
- `tp3 system1 package-delivery` arma un zip chico con solo el motor, como pide la entrega.

Punto fino para la defensa: el deck actual usa como fuente de verdad `artifacts/system1/studies/inciso-1.1`, mientras que el config de ejemplo del repo sirve para demo y verificación rápida.

2.10. Cómo calculamos cada observable

Inciso 1.1

- Medimos tiempo de pared real del motor completo a horizonte físico fijo $t_f = 500$ s.
- El tiempo reportado incluye también la escritura de snapshots.

Inciso 1.2

- Para cada realización preservamos toda la serie $C_{fc}(t)$.
- Ajustamos una recta por OLS en tiempo físico.
- La pendiente de esa recta es J .

Inciso 1.3

- Tomamos la historia de $F_u(t)$ en snapshots irregulares.
- La remuestreamos con retención de orden cero usando $\Delta t = 0.5$ s.
- Declaramos estacionario si la diferencia entre medias de dos ventanas consecutivas de 10 s es menor o igual a 0.02, repetido en 3 chequeos.
- Una vez detectado, corremos 20 s más para medir F_{est} .

Inciso 1.4

- Las capas tienen ancho fijo $dS = 0.2$ m.
- Solo contamos partículas frescas entrantes, o sea con $\mathbf{R} \cdot \mathbf{v} < 0$.

- La densidad se promedia sobre tiempos post-estacionarios.
- La velocidad normal se promedia solo cuando hay partículas válidas en la capa, sin inventar ceros.
- El flujo se define como $J_{in}(S) = \langle \rho_f^{in} \rangle(S) |\langle v_f^{in} \rangle(S)|$.

2.11. Limitaciones que conviene reconocer antes de que las marquen

- El dataset que estamos presentando tiene **2 realizaciones por valor de N** . Alcanza para mostrar tendencias gruesas, no para hacer afirmaciones finas.
- El tiempo al estacionario aparece siempre en 59 s **en esta base de datos y con este protocolo**. No debe venderse como ley universal del sistema.
- El motor no modela de manera explícita colisiones perfectamente simultáneas de tres o más partículas.
- El crecimiento del runtime en N es muy fuerte; por eso no conviene improvisar una corrida pesada en vivo.

2.12. Frases a evitar

- “Hay más vecinos” si no lo respaldan con una métrica específica de vecindad.
- “El sistema se ordena” o “se congestiona” si no muestran un observable que mida eso.
- “La tasa de escaneo aumenta con N ” a secas, porque en los datos mostrados sube hasta $N = 750$ y luego baja en $N = 1000$.
- “Llega siempre a estacionario en 59 s” como afirmación universal.
- “El gráfico demuestra causalidad” cuando en realidad solo muestra correlación o tendencia descriptiva.

3. Guion de la presentación

3.1. Ruta sugerida para 13 minutos

La estrategia recomendada es:

1. Introducción muy corta.
2. Implementación fuerte y clara.
3. Definición de observables sin saltar la parte matemática.
4. Resultados con lectura directa de cada figura.
5. Conclusiones sobrias y ligadas a las diapositivas mostradas.

3.2. Slide por slide

Portada

Tiempo sugerido: 15–20 s

- **Idea central:** Decir qué sistema resolvieron y qué van a mostrar.
- **Qué decir:** “Vamos a presentar el Sistema 1 del TP3: un motor dirigido por eventos para partículas rígidas en un recinto circular con obstáculo central, junto con el estudio pedido en los incisos 1.1 a 1.4.”

Objetivo y motivación

Tiempo sugerido: 35–40 s

- **Idea central:** Justificar la elección del enfoque y qué observables interesan.
- **Qué decir:** “Nuestro objetivo fue pasar el modelo del enunciado a un simulador event-driven y medir cómo esa dinámica microscópica se refleja en observables macroscópicos como la tasa de escaneo, la fracción usada estacionaria y los perfiles radiales.”

Sistema físico y reglas del modelo

Tiempo sugerido: 45–55 s

- **Idea central:** Dejar fija la geometría y la máquina de estados.
- **Qué decir:** “El sistema es un anillo: pared circular exterior, obstáculo fijo central y partículas de radio y masa unidad. Todas empiezan frescas; tocar el obstáculo las vuelve usadas y tocar la pared exterior las vuelve a frescas.”

Representación computacional del modelo

Tiempo sugerido: 40–50 s

- **Idea central:** Mostrar que hay una traducción limpia entre modelo y software.
- **Qué decir:** “En código cada partícula guarda posición, velocidad y estado. A partir de ese estado predecimos choques futuros, los ordenamos en una cola priorizada, resolvemos el siguiente evento y registramos observables.”

Algoritmo dirigido por eventos y colisiones

Tiempo sugerido: 60–75 s

- **Idea central:** Explicar el loop canónico.
- **Qué decir:** “El motor no avanza por frames. Extrae el próximo evento válido, mueve todo el sistema exactamente hasta ese tiempo, resuelve la colisión y vuelve a programar solo los eventos afectados. Para manejar eventos viejos usamos invalidación diferida sobre la cola.”

Traducción del modelo a código

Tiempo sugerido: 45–55 s

- **Idea central:** Dar el detalle matemático mínimo que muestra entendimiento.
- **Qué decir:** “Los tiempos de choque salen de ecuaciones de contacto. En partícula-partícula resolvemos la primera raíz positiva de la cuadrática; en pared y obstáculo resolvemos la intersección con la frontera circular y validamos el sentido físico del choque.”

Parámetros del estudio

Tiempo sugerido: 45–55 s

- **Idea central:** Separar claramente configuración fija y variable.
- **Qué decir:** “Para el estudio fijamos geometría y protocolo, variamos N y repetimos realizaciones. Para 1.3 y 1.4 remuestreamos $F_u(t)$ con $\Delta t = 0.5$ s, detectamos estacionario por ventanas y extendemos 20 s para medir observables estacionarios.”

Visualización de la corrida

Tiempo sugerido: 25–35 s

- **Idea central:** Usar la animación como apoyo, no como relleno.
- **Qué decir:** “La animación sale del mismo archivo textual del motor. Verde representa fresca, violeta representa usada. Esto nos sirve para corroborar visualmente la dinámica, pero las conclusiones cuantitativas salen de las series y perfiles agregados.”

Definición de observables

Tiempo sugerido: 60–75 s

- **Idea central:** Tomarse el tiempo para que el resto de las figuras se entiendan.
- **Qué decir:** “Definimos $C_{fc}(t)$ como los contactos acumulados fresca a usada; la pendiente de su ajuste lineal nos da J . Para la fracción usada usamos $F_u(t) = N_u(t)/N$ y medimos tanto el tiempo al estacionario como su promedio estacionario. Para perfiles radiales filtramos solo partículas frescas entrantes.”

1.1 Tiempo de ejecución vs N

Tiempo sugerido: 40–50 s

- **Idea central:** Leer tendencia y magnitudes.
- **Qué decir:** “El costo computacional crece muy fuerte con N . En esta base de datos pasamos de alrededor de 4.61 s en $N = 250$ a casi 1980 s en $N = 1000$, con dispersión mayor en los regímenes más densos.”

1.2 Evoluciones típicas de $C_{fc}(t)$

Tiempo sugerido: 35–45 s

- **Idea central:** Mostrar cómo se obtiene J a nivel realización.
- **Qué decir:** “Estas dos corridas representativas muestran que el observable base es una serie acumulada. El ajuste lineal sobre toda la serie nos da una pendiente y esa pendiente es la tasa de escaneo de esa realización.”

1.2 Tasa de escaneo promedio

Tiempo sugerido: 40–50 s

- **Idea central:** Evitar vender monotonidad que no está.
- **Qué decir:** “Al promediar por N vemos que J no resulta monótona en esta grilla: aumenta hasta un máximo medido en $N = 750$ y luego cae en $N = 1000$. Esa es la lectura segura de estos datos.”

1.3 Evoluciones típicas de $F_u(t)$

Tiempo sugerido: 35–45 s

- **Idea central:** Conectar el observable con el protocolo estacionario.
- **Qué decir:** “La línea vertical punteada marca el instante en que el protocolo detecta estacionario; la horizontal marca el promedio post-estacionario. No tomamos ese valor a ojo: sale de una regla fija aplicada a todas las realizaciones.”

1.3 Métricas estacionarias

Tiempo sugerido: 40–50 s

- **Idea central:** Marcar alcance y límite del dataset.
- **Qué decir:** “En esta base de datos el tiempo detectado al estacionario es 59 s para todas las realizaciones. Además, F_{est} alcanza su mayor valor medido en $N = 100$ y luego decrece para N más grandes.”

1.4 Perfil radial representativo

Tiempo sugerido: 35–45 s

- **Idea central:** Explicar qué se está promediando.
- **Qué decir:** “Acá vemos, para un N representativo, la densidad de partículas frescas entrantes, su velocidad normal media y el flujo derivado, todos promediados en el régimen post-estacionario.”

1.4 Capa cercana a $S \approx 2$

Tiempo sugerido: 40–50 s

- **Idea central:** Leer solo lo que el gráfico habilita.
- **Qué decir:** “Al mirar la primera capa accesible, $[2.0, 2.2)$, no observamos señal medible para $N = 10$ y $N = 50$. A partir de $N = 100$ aparecen densidad, velocidad y flujo distintos de cero, con un máximo medido en $N = 750$.”

Conclusiones generales

Tiempo sugerido: 45–55 s

- **Idea central:** Cerrar con dos o tres mensajes fuertes.
- **Qué decir:** “La primera conclusión es computacional: el costo crece muy fuerte con N . La segunda es física: la tasa de escaneo y la fracción usada estacionaria no muestran una respuesta monótona simple en toda la grilla. La tercera es metodológica: el mismo motor sirvió para generar snapshots, animaciones, series y perfiles.”

Conclusiones sobre la zona próxima al obstáculo

Tiempo sugerido: 30–40 s

- **Idea central:** Cerrar sin sobregeneralizar.
- **Qué decir:** “En la primera capa accesible, las magnitudes cerca del obstáculo solo aparecen a partir de cierto tamaño del sistema y el mayor flujo medido en esta grilla ocurre en $N = 750$. Dado que trabajamos con dos realizaciones por N , tomamos esto como tendencia observada y no como ley definitiva.”

3.3. Plan B si los cortan de tiempo

Si los apuran, prioricen en este orden:

1. Objetivo y sistema físico.
2. Dos slides de implementación: algoritmo + traducción a código.
3. Definición de observables.
4. Runtime vs N .
5. Tasa de escaneo promedio.
6. Métricas estacionarias.
7. Capa cercana a $S \approx 2$.
8. Conclusiones.

4. Preguntas y respuestas posibles

4.1. Implementación del motor

1. ¿Por qué eligieron simulación dirigida por eventos y no un Δt fijo?

Porque el modelo cambia solo en colisiones. Un Δt fijo introduce un parámetro numérico artificial y obliga a balancear costo contra precisión. Con event-driven avanzamos exactamente a tiempos de choque.

2. ¿Qué guarda exactamente un evento?

Guarda el tiempo predicho, el tipo de evento, las partículas involucradas y el contador de colisiones de esas partículas al momento de programarlo. Eso permite decidir luego si el evento sigue siendo válido.

3. ¿Qué es invalidación diferida?

Es dejar eventos viejos dentro del heap y descartarlos cuando salen a la cima si alguno de sus contadores ya no coincide. Es más simple que intentar borrar eventos arbitrarios adentro de la cola.

4. ¿Por qué recalculan solo choques de partículas afectadas?

Porque solo esas partículas cambiaron velocidad o estado. El resto del sistema sigue con la misma trayectoria balística prevista, así que no hace falta recomputar todo el futuro.

5. ¿Cómo calculan el choque con la pared o con el obstáculo?

Resolviendo cuándo la trayectoria $\mathbf{r} + \mathbf{v}t$ llega al radio de viaje correspondiente. Luego se filtra la raíz para quedarse con la que representa realmente un impacto hacia la frontera adecuada.

6. ¿Qué pasa si la inicialización aleatoria por rechazo falla?

El motor cae a una inicialización de respaldo sobre anillos, con posiciones candidatas distribuidas en la región accesible y luego mezcladas aleatoriamente. Eso evita fallos por densidad o mala suerte del rechazo.

7. ¿El color por qué sale del simulador y no del GIF?

Porque el enunciado pide color en la salida textual. El GIF es un consumidor de esa salida, no la fuente de verdad del estado.

8. ¿Qué no modela este motor?

No modela explícitamente colisiones perfectamente simultáneas de tres o más partículas. El esquema supone colisiones instantáneas tratadas de a un evento por vez.

4.2. Análisis y resultados

9. ¿Cómo calcularon la tasa de escaneo J ?

Para cada realización guardamos toda la serie $C_{fc}(t)$ y ajustamos una recta por OLS en tiempo físico. La pendiente del ajuste es J .

10. ¿Por qué usan todo $C_{fc}(t)$ y no solo el final?

Porque el enunciado pide ajustar una recta a la evolución temporal. Usar toda la serie aprovecha la información de la trayectoria completa del observable, no solo un punto final.

11. ¿Cómo detectan estacionario en $F_u(t)$?

Remuestreamos la serie con retención de orden cero en una grilla uniforme de 0.5 s. Comparamos medias de dos ventanas consecutivas de 10 s y pedimos tres chequeos consecutivos con diferencia menor o igual a 0.02.

12. ¿Por qué en su dataset el tiempo estacionario vale 59 s para todos los N ?

La respuesta segura es: eso es lo que dio esta base de datos con este protocolo de detección. No lo presentamos como una propiedad universal del sistema, sino como resultado de las realizaciones mostradas.

13. ¿Por qué solo dos realizaciones por N ?

Porque esta presentación se construyó sobre la base de datos ya generada en `artifacts/system1/studies/inciso-1.1`. Sirve para exhibir el pipeline y tendencias gruesas, pero reconocemos que un estudio final más robusto debería aumentar repeticiones.

14. ¿Por qué la capa “cerca de $S = 2$ ” la fijaron en $[2.0, 2.2]$?

Porque el ancho de capa es $dS = 0.2\text{m}$ y el primer radio accesible para el centro de una partícula es $r_0 + r = 2\text{m}$. Entonces la primera capa accesible es precisamente $[2.0, 2.2]$.

15. ¿Por qué promedian densidad y velocidad de manera distinta?

Porque son magnitudes distintas. La densidad sí puede contribuir con cero cuando una capa queda vacía. En cambio, para la velocidad normal no tiene sentido meter ceros artificiales cuando no hubo partículas válidas; solo se promedia sobre muestras reales.

16. ¿Por qué en los gráficos de la presentación no unieron los puntos con líneas?

Porque esos plots son un barrido sobre valores discretos de N . La línea puede sugerir una interpolación continua o una tendencia más fuerte de la que realmente se midió. Para la presentación preferimos marcadores y barras de error.

17. ¿Qué conclusión fuerte sí pueden sostener con estos datos?

Que el runtime crece muy fuerte con N , que la tasa de escaneo no es monótona en la grilla mostrada y que cerca del obstáculo no hay señal medible para los N más chicos de esta base.

18. ¿Qué conclusión no deberían sostener?

No conviene afirmar mecanismos causales que no midieron, por ejemplo “hay más vecinos” o “aparece orden colectivo”, porque no hay un observable en esta presentación que lo pruebe.

4.3. Uso responsable de IA y validación

19. ¿Usaron IA?

Una respuesta responsable es: “Sí, como asistencia para acelerar escritura y organización, pero no como caja negra. Validamos el modelo contra el enunciado y la wiki del repo, revisamos el código crítico del motor, y podemos explicar cómo se calcula cada choque, cada observable y cada figura.”

20. ¿Cómo demuestran que entendieron lo que hace el simulador?

Mostrando que pueden explicar el loop event-driven, cómo se validan eventos obsoletos, cómo se calcula J , cómo se detecta estacionario y ejecutando una corrida corta en vivo si hace falta.

21. ¿Cómo saben que el resultado no está “inventado” por la IA?

Porque las salidas se contrastaron con los contratos del repo: comandos reales, snapshots reales, CSVs agregados y figuras generadas a partir de esos datos. Además, los puntos críticos del algoritmo quedan verificables leyendo el código y ejecutando ejemplos pequeños.

22. ¿Si yo les cambio un parámetro en vivo, qué deberían poder anticipar?

Deberían poder anticipar al menos efectos cualitativos: cambiar N impacta el número de colisiones y el runtime; cambiar `snapshot_every` afecta la cantidad de snapshots y el peso de salida; cambiar el horizonte temporal afecta cuánto observable acumulado se registra.

23. ¿Qué parte del trabajo fue más importante validar?

La predicción y resolución de choques, porque de eso depende todo lo demás. Después, la definición exacta de observables y el protocolo estacionario, porque son los que transforman la simulación en resultados medibles.

5. Uso responsable de IA

Si el profesor pregunta explícitamente por IA, conviene responder sin ponerse a la defensiva y sin fingir que no se usó.

Respuesta corta sugerida

“Usamos IA como herramienta de asistencia, pero no delegamos la comprensión del problema. Lo que validamos manualmente fue: el modelo físico del enunciado, la traducción a simulación dirigida por eventos, la definición de los observables, el protocolo estacionario y la ejecución real del simulador. Por eso podemos explicar qué hace cada parte y correr una simulación corta si hace falta.”

Qué evidencia concreta pueden mencionar

- El repo tiene una wiki local que fija el contrato del sistema, los observables y el protocolo experimental.
- Los comandos del CLI existen y se pueden ejecutar: `run`, `animate`, `study`, `package-delivery`.
- La presentación no sale de texto inventado: sale de los CSVs y series guardados bajo `artifacts/system1/studies/inciso-1.1`.
- Pueden explicar cómo se obtiene un tiempo de choque, cómo se invalida un evento viejo y cómo se calcula el promedio estacionario.

Qué no conviene responder

- “La IA hizo el código y nosotros después lo probamos.”
- “No sabemos exactamente por qué funciona pero da bien.”
- “Lo importante es el resultado, no tanto el proceso.”

6. Checklist para una demostración en vivo

6.1. Secuencia recomendada

La demo más segura no es correr un `study` pesado. Lo recomendable es:

1. validar config,
2. correr una simulación corta ya configurada,
3. si lo piden, animar ese archivo textual.

6.2. Comandos

Ejecutar desde la raíz del repo:

```
PYTHONPATH=src python3 -m tp3_sds system1 validate-config \
--config configs/system1.example.toml
```

```
PYTHONPATH=src python3 -m tp3_sds system1 run \
--config configs/system1.example.toml
```

```
PYTHONPATH=src python3 -m tp3_sds system1 animate \
--input artifacts/system1/example_run.txt \
--output artifacts/system1/example_run.gif \
--fps 20 \
--playback-duration 8
```

6.3. Qué deberían poder comentar si corre bien

- Se genera un `.txt` autocontenido con header, snapshots y partículas.
- El CLI informa eventos procesados, cantidad de snapshots y scanning count.
- La animación se construye a partir del `.txt`, no desde un renderer acoplado al motor.

6.4. Qué no conviene hacer en vivo

- Correr un `study` con N grandes.
- Compilar la presentación delante del profesor.
- Cambiar código sobre la marcha.

7. Checklist final antes de exponer

- Reemplazar placeholders de portada en el deck.
- Tener a mano el PDF de la presentación y este PDF de guion.
- Decidir quién explica implementación, quién explica resultados y quién absorbe preguntas técnicas.
- Acordar una respuesta consistente sobre uso responsable de IA.
- Practicar una versión de 11 minutos y una versión de 13 minutos.
- Tener abierta una terminal ya ubicada en la raíz del repo.

Cierre recomendado

La defensa tiene que dar esta impresión: **el grupo entiende el motor, entiende el análisis y sabe qué parte de lo que muestra está firmemente respaldada por datos y cuál parte necesita más repeticiones para ser una conclusión fuerte.**